

Analisi in tempo reale di dati sui voli con Apache Flink

Sistemi e Architetture per Big Data - A.A. 2025/26

Andrea Ferretti

Dipartimento di Ingegneria Civile e Ingegneria Informatica

Università degli Studi di Roma Tor Vergata

Roma, Italia

andrea.ferretti@students.uniroma2.edu

Sommario—Il presente documento descrive l'implementazione e la valutazione di un'applicazione di stream processing basata su Apache Flink per l'analisi in tempo reale di eventi relativi a voli aerei operati negli Stati Uniti. Il sistema è in grado di ingerire eventi storici, riproducendoli in streaming, gestire il tempo logico degli eventi e calcolare statistiche su finestre temporali, tra le quali il monitoraggio operativo delle compagnie e l'individuazione di aeroporti critici. Sono inoltre analizzate le metriche prestazionali dell'infrastruttura, quali latenza e throughput, al variare del livello di parallelismo.

Index Terms—Apache Flink, Stream Processing, Big Data, Data Ingestion

I. INTRODUZIONE

Nel settore dei trasporti aerei è importante identificare tempestivamente situazioni di congestione e misurare metriche critiche, per agire in tempi rapidi. A differenza dell'analisi batch, che processa i dati al termine della fase di raccolta, l'analisi streaming elabora gli eventi in tempo reale, non appena questi sono prodotti.

L'implementazione di un'architettura distribuita in grado di eseguire l'analisi dei dati in tempo reale presenta diverse sfide, quali la gestione di operatori con stato, l'implementazione di finestre temporali e la gestione di eventi fuori ordine. Quest'ultimo aspetto richiede l'utilizzo di diverse strategie che comportano un compromesso tra latenza di processamento e accuratezza delle metriche misurate.

Lo scopo del progetto è produrre quanto richiesto da due query relative a uno stream di eventi di voli statunitensi, utilizzando Apache Flink. In particolare, l'applicazione Flink riceve in input lo stream, assegna a ciascun evento in esso contenuto un timestamp logico basato sull'event time, calcola delle statistiche su finestre temporali, gestendo eventi fuori ordine, e salva i risultati sia in formato csv, sia su InfluxDB, per consentirne la visualizzazione tramite Grafana.

II. ARCHITETTURA DEL SISTEMA

L'architettura del sistema è stata progettata per garantire l'elaborazione continua degli eventi di volo, strutturando una pipeline end-to-end. Il sistema si articola logicamente in tre macro-livelli (moduli) interconnessi: il livello di data ingestion, il livello di stream processing e il livello di data serving.

Il primo modulo simula un flusso continuo a partire dal dataset, garantendo il corretto riordinamento temporale e l'emissione controllata degli eventi. L'elaborazione dei dati è delegata ad Apache Flink (secondo modulo), un framework per il processamento di dati di tipo streaming. Infine, il modulo di data serving si occupa di scrivere i risultati contemporaneamente in formato CSV e su InfluxDB, data store per serie temporali, al quale accede Grafana per mostrare l'evolversi delle statistiche nel tempo. I tre moduli sono interconnessi tramite Apache Kafka, il quale ricopre il ruolo di message broker, consentendo di disaccoppiare le tecnologie adottate e di gestire i dati in modo scalabile. I moduli, implementati tramite il framework di virtualizzazione Docker, sono orchestrati da Docker Compose e sono eseguiti in modalità standalone.

A. Replay e Accelerazione Temporale

L'engine di generazione dello stream è stato implementato con il linguaggio Python, il quale offre Pandas, una libreria di processamento di tabelle di dati, che facilita e velocizza la scrittura del codice, evitando cicli for.

Inizialmente, l'engine carica il dataset, suddiviso in quattro file csv, e li unifica in un unico Dataframe di Pandas. Successivamente, viene generato l'event time per ciascun volo combinando i campi relativi ad anno, mese, giorno e orario schedato di partenza. Tale timestamp viene convertito in formato Unix time ed espresso in secondi; l'unità di misura è stata scelta per semplificare il calcolo del fattore di accelerazione temporale. Il dataset viene infine ordinato cronologicamente per garantire l'emissione sequenziale degli eventi.

Al fine di simulare un flusso continuo e gestire i dati in tempi ragionevoli, è stato applicato un meccanismo di accelerazione temporale. Il fattore di accelerazione è calcolato dinamicamente come rapporto tra l'estensione temporale reale dei dati e la durata target prefissata, pari a 3 minuti, determinati come compromesso tra l'esigenza di avere uno stream di dimensioni significative e il vincolo di svolgere gli esperimenti in un tempo ragionevole. Calcolando il rapporto tra la differenza temporale di due eventi successivi e il fattore di accelerazione, il sistema ridimensiona proporzionalmente le attese (implementate come sleep del processo) tra l'invio di messaggi successivi, mantenendo inalterate le distanze temporali relative tra i voli storici.

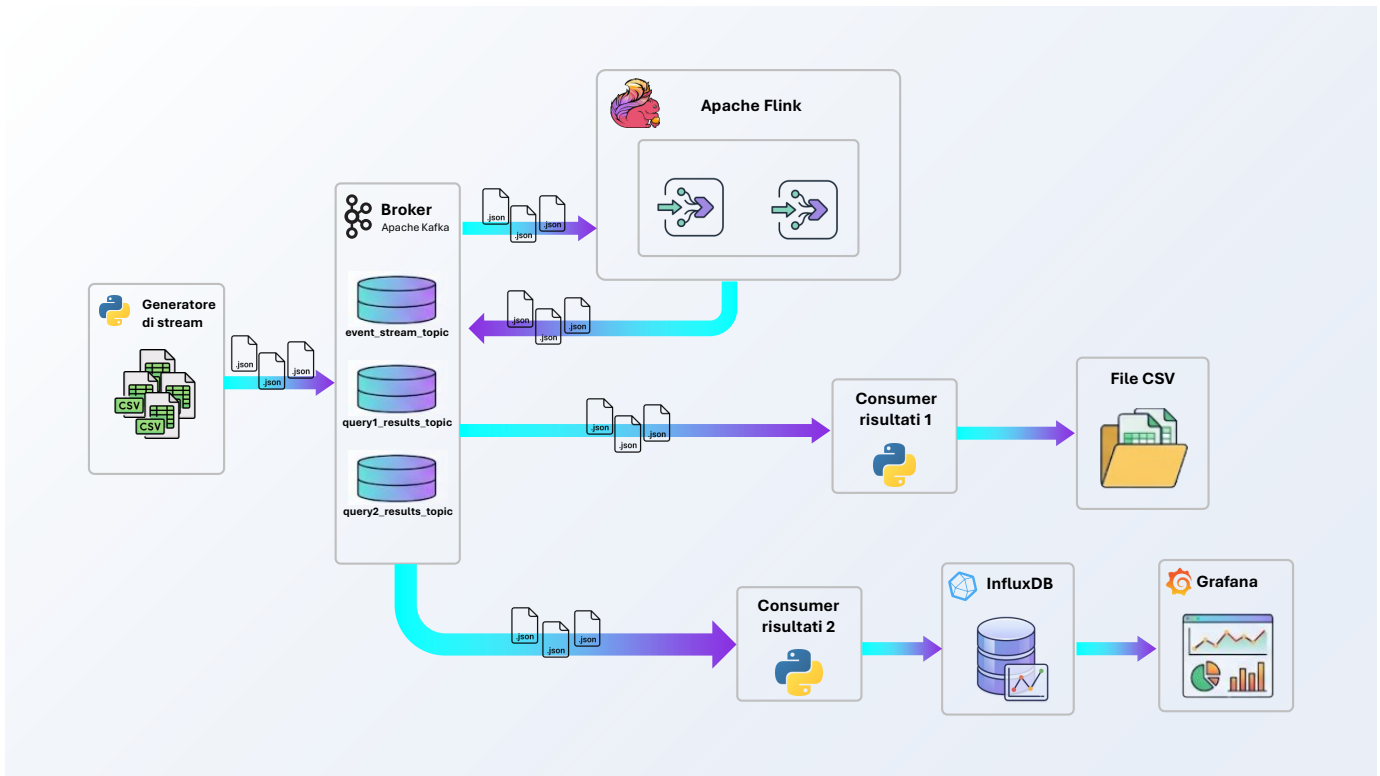


Figura 1. Panoramica dell'architettura di sistema per l'ingestione, l'elaborazione e la visualizzazione degli eventi.

B. Meccanismo di Ingestion

Il livello di data ingestion è implementato tramite Apache Kafka, che svolge il ruolo di strato di disaccoppiamento e buffer di memoria tra il generatore degli eventi e l'app di stream processing.

Al momento dell'emissione verso il message broker, il producer converte l'event time in millisecondi e lo imposta direttamente come timestamp nativo del messaggio Kafka. Questa scelta alleggerisce il carico elaborativo a valle: Apache Flink, infatti, non deve effettuare il parsing dell'intero payload e calcolare il tempo logico, ma utilizza i metadati di Kafka per l'assegnazione dei timestamp e l'avanzamento dei watermark.

Per quanto riguarda il deployment, Apache Kafka è stato eseguito istanziando un solo container e quattro partizioni del topic. Questa scelta è motivata dall'esigenza di parallelizzare la lettura dei dati, facendo corrispondere il numero di partizioni al numero massimo di task di lettura previsti negli esperimenti.

C. Stream processing

Il nucleo di processamento dei dati della pipeline è affidato ad Apache Flink. Dal punto di vista infrastrutturale, il deployment avviene in Application Mode: il cluster è istanziato in modalità standalone-job esclusivamente per eseguire l'applicazione, scritta con il linguaggio di programmazione Java e memorizzata tramite fat jar in un'immagine Docker personalizzata generata attraverso una build multi-stage basata su Maven.

L'infrastruttura di calcolo è orchestrata da un JobManager, responsabile della coordinazione e della traduzione del DAG, ed è composta da un singolo TaskManager per l'esecuzione del job. Il numero di task slot impostati per il TaskManager è 2.

Il job interagisce con Kafka sia in fase di lettura da sorgente che di scrittura tramite sink. Per consentire ciò, è stato utilizzato il connettore nativo offerto da Flink. La serializzazione e la deserializzazione dei dati, sia in scrittura che in lettura, sono state implementate attraverso classi custom che utilizzano l'object mapper di Jackson per effettuare le conversioni usando il formato JSON. Tale formato è stato preferito ad Avro per via della maggiore facilità di implementazione: il formato Avro, infatti, richiede la progettazione di un Service Registry a cui è necessario accedere per salvare e leggere lo schema dei dati.

D. Data serving

Il modulo di data serving rappresenta il livello terminale dell'architettura. Da un lato le metriche prodotte da Apache Flink vengono salvate su file system locale sotto forma di file CSV. Dall'altro, al fine di visualizzare le metriche in tempo reale tramite Grafana, i risultati vengono scritti su InfluxDB, un database specificamente ottimizzato per l'ingestione ad alta frequenza di serie temporali.

A livello di deployment, sono stati utilizzati due container Python, uno per gestire la persistenza dei dati su filesystem locale in formato CSV e uno per inviare i dati a InfluxDB. Entrambi i container implementano un consumer che legge i

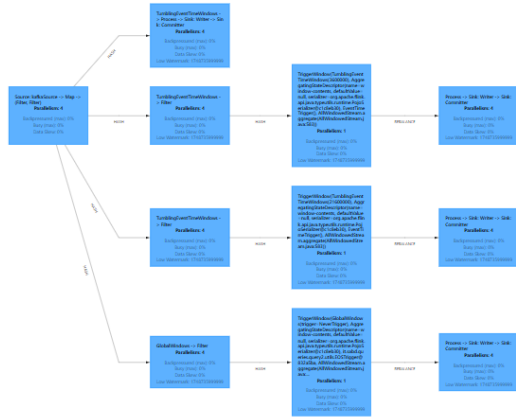


Figura 2. DAG di esecuzione del job Flink

risultati dal broker Kafka e li deserializza dal formato JSON. Il consumer che si occupa di salvare i risultati in file CSV si limita a formattare i risultati in stringhe e a salvarle su disco. Il consumer di InfluxDB utilizza la libreria client di InfluxDB per creare oggetti Point da salvare e gestire tag, fields e measurement.

L'ecosistema è completato da un'istanza containerizzata di InfluxDB e una di Grafana. Quest'ultima, interfacciandosi direttamente con InfluxDB, esegue query continue per alimentare una dashboard interattiva, garantendo l'esplorazione visiva e dinamica delle metriche, con latenze molto basse per via delle proprietà del data store, il quale consente di estrarre dati su finestre temporali e definire diversi tag indicizzati per velocizzare le query.

III. LOGICA DI STREAM PROCESSING

A. Gestione del Tempo e Watermarking

La gestione di eventi fuori ordine è cruciale in sistemi di stream processing distribuiti. Dal momento che (i) lo stream engine invia i dati a Kafka in ordine di event time, (ii) il cluster è eseguito in modalità standalone e pertanto non ci sono perdite di pacchetti, i dati all'interno di ciascuna partizione sono ordinati. Non essendo quindi presenti eventi in ritardo nella singola partizione, è necessario gestire unicamente l'ordinamento degli eventi tra le partizioni. A tal fine, è stata utilizzata la strategia di watermarking "forMonotonousTimestamps" per ciascuna partizione, di conseguenza il watermark generato dal source Kafka è pari al minimo dei watermark generati da ciascuna partizione. Non è stato inoltre necessario gestire situazioni di idleness delle partizioni, in quanto ciascuna di esse riceve il messaggio di "end of stream".

B. Pulizia e Pre-processamento

All'interno del job Flink non è stata implementata una reale fase di pre-processamento: nella fase di deserializzazione è stato definito un POJO con i soli campi necessari per eseguire le due query, popolati tramite la classe ObjectMapper di Jackson. I valori nulli non sono imputati perchè rappresentano

una percentuale molto piccola rispetto al numero totale di record. Pertanto, si è scelto di non considerarli nel calcolo delle metriche, filtrandoli localmente. Questo approccio è stato adottato in particolare per calcolare le statistiche che coinvolgono il ritardo in partenza, evitando così di introdurre distorsioni statistiche da imputazioni non rappresentative.

C. Sviluppo delle Query

a) *Query 1*: La prima query ha l'obiettivo di monitorare in tempo reale lo stato operativo di un sottoinsieme specifico di compagnie aeree statunitensi: American Airlines (AA), Delta (DL), United (UA) e Southwest (WN).

A livello implementativo, il flusso di eventi in ingresso viene inizialmente sottoposto a un filtraggio preliminare per isolare esclusivamente i record appartenenti ai quattro vettori di interesse. Successivamente, lo stream viene partizionato logicamente per compagnia aerea sfruttando l'operatore *keyBy*. Questa scelta permette ad Apache Flink di parallelizzare l'elaborazione, mantenendo stati separati e indipendenti per ciascun vettore.

Sullo stream partizionato vengono quindi applicate delle finestre temporali basate su event time di tipo tumbling della durata di un'ora.

All'interno di ciascuna finestra, una *AggregateFunction* elabora le tuple in ingresso per calcolare le metriche richieste. La *AggregationFunction* consente di minimizzare lo stato della finestra aggiornando i suoi attributi ad ogni arrivo di un nuovo evento. Nello specifico, lo stato tiene traccia dei seguenti indicatori:

- il contatore del numero totale di voli osservati nella finestra corrente;
- i contatori del numero totale di voli completati, cancellati e devianti;
- la somma dei ritardi in partenza e il contatore degli eventi non cancellati e con ritardo non nullo
- il contatore del numero di voli non cancellati e con ritardo in partenza strettamente maggiore di quindici minuti

Al momento della chiusura della finestra temporale, vengono calcolate le statistiche finali: il valor medio del ritardo in partenza, calcolato come il rapporto tra la somma dei ritardi e il contatore degli eventi non cancellati e con ritardo non nullo, il tasso di cancellazione, calcolato come percentuale di voli cancellati rispetto al volume totale dei voli nella finestra e il tasso di partenze in ritardo, definito come la percentuale di voli non cancellati che presentano un ritardo in partenza maggiore di quindici minuti. La *AggregateFunction* è passata alla funzione *aggregate* assieme a una *ProcessWindowFunction*. Questa funzione consente di accedere ai metadati della finestra e, in particolare, alla chiave e ai timestamp di inizio e di fine.

Infine, i dati sono serializzati con un serializzatore custom e sono scritti nel broker Kafka, in un apposito topic.

b) *Query 2*: La seconda query ha l'obiettivo di identificare in tempo reale gli aeroporti di partenza maggiormente interessati da ritardi significativi. A tal fine, viene introdotto il concetto di "ritardo significativo", definito come un ritardo

in partenza superiore a 30 minuti per voli che non sono stati cancellati o deviati.

Lo stream di eventi viene dapprima filtrato selezionando esclusivamente i voli che non sono stati cancellati o deviati, e successivamente viene partizionato logicamente per aeroporto di origine tramite l'operatore *keyBy*. A partire da questo stream, il DAG logico si ramifica in tre flussi paralleli e indipendenti, al fine di calcolare le statistiche su finestre temporali diverse: finestre tumbling da 1 ora, finestre tumbling da 6 ore e una finestra globale per cumulare le statistiche dall'inizio del dataset. A livello implementativo, si è scelto di modellare la finestra globale utilizzando una window della durata di 4 mesi, per la quale è stato implementato un trigger custom attivato in corrispondenza del watermark generato in seguito alla lettura dei messaggi di appositi messaggi di "end of stream" immessi in ciascuna partizione del topic dallo stream engine. Lo stream engine imposta l'event time di tali messaggi al 1 giugno 2025, data successiva al 30 aprile 2025.

La scelta di ramificare il job in tre flussi diversi, assieme all'utilizzo della tumbling window per la finestra globale, consente di applicare le stesse concatenazioni degli operatori. All'interno di ciascuna partizione, una *AggregateFunction* aggiorna costantemente lo stato della finestra, composto da:

- il numero di voli non cancellati e non deviati;
- il numero di voli con ritardo significativo in partenza;
- la somma cumulativa dei ritardi in partenza e un contatore dei voli che non sono stati nè cancellati nè deviati, e con ritardo in partenza non nullo;
- il valore massimo del ritardo in partenza osservato;
- i primi venti voli con ritardo in partenza significativo, ordinati per ritardo in partenza. Per ciascun volo sono riportati la compagnia aerea, l'aeroporto di destinazione e il ritardo in partenza.

Al fine di limitare la memoria occupata dallo stato nell'aggiornamento, la classifica è aggiornata ad ogni nuovo arrivo dell'evento. Per garantire alte prestazioni nell'aggiornamento, è stata utilizzata una *PriorityQueue*, in particolare un min-heap dove la chiave di ordinamento è il ritardo in partenza. Il min-heap consente di accedere al minimo con complessità computazionale $O(1)$ e di inserire un elemento con complessità $O(\log(n))$. Se la coda è piena, all'arrivo di un nuovo evento con ritardo in partenza significativo, viene effettuato il confronto tra la chiave di ordinamento dell'evento e quella del minimo. Se l'evento ha un valore maggiore del minimo, la coda viene aggiornata con l'inserimento dell'evento e la rimozione del minimo. Se la coda non è piena, l'evento viene aggiunto senza effettuare controlli.

Al momento della valutazione della finestra, la *AggregateFunction* calcola i tassi richiesti e converte la coda in una lista ordinata. Inoltre, viene eseguita anche una *ProcessWindowFunction* che accede al timestamp di inizio della finestra e alla chiave.

Successivamente, per poter calcolare la classifica sugli aeroporti, è necessario convogliare il flusso partizionato in un unico flusso, in modo da avere una visione globale dei dati. Dapprima si selezionano gli aeroporti che hanno un numero di

voli non cancellati e non deviati maggiore o uguale a trenta, quindi si crea una finestra di tipo tumbling avente la stessa durata della finestra precedente tramite l'operatore *windowAll*. Lo stato di tale finestra è aggiornato in corrispondenza dell'arrivo di un nuovo evento tramite una *AggregateFunction*. Anche in questo caso, dovendo calcolare una classifica in ordine decrescente, viene utilizzato un min-heap come stato intermedio, il quale è poi convertito in una lista ordinata.

Infine, il flusso scrive i dati tramite un sink Kafka e un serializzatore JSON custom sul broker Kafka. Ciascuno dei tre flussi logici scrive i dati in un topic diverso, in modo da facilitare la lettura e la gestione dei dati da parte dei consumer.

IV. VALUTAZIONE SPERIMENTALE

A. Ambiente di Test

Il test è stato eseguito su una istanza EC2 con 4 vCPU, 16GB di RAM e 32GB di disco. Le metriche sono state raccolte collegando Flink e InfluxDB.

B. Throughput e Latenza

Il throughput è stato calcolato come il numero di record al secondo in uscita da Sink Writer. La misurazione della latenza è stata implementata mediante codice, assegnando a ciascun evento il valore del clock della macchina al momento della sua uscita dal source Kafka e determinando per ciascuna finestra il minimo di tali valori, così da rilevare l'intero periodo temporale trascorso dal primo evento all'interno alla finestra. Prima di inviare l'evento ai Sink si calcola la differenza tra il clock della macchina e quello associato all'evento stesso.

C. Impatto del Parallelismo

L'esperimento è stato svolto utilizzando un task manager con 4 task slots e gradi di parallelismo 1, 2 e 4. Si riportano di seguito i valori medi e l'deviazione standard di throughput e latenza rilevati in corrispondenza dei diversi gradi di parallelismo, eseguendo 5 volte le rilevazioni per ciascuno di essi.

Tabella I
THROUGHPUT E LATENZA AL VARIARE DEL GRADO DI PARALLELISMO

Grado di parallelismo	Throughput(ev/s)	Latenza(s)
1	12.233 ± 30,91	27 ± 0,03
2	12.193 ± 40,27	13,8 ± 0,04
4	12.176 ± 23,57	7 ± 0,01

All'aumentare del grado di parallelismo si osserva una modesta riduzione del throughput, probabilmente imputabile a un overhead di comunicazione e di suddivisione degli eventi tra i task paralleli. Al tempo stesso, si osserva una riduzione della latenza, indicativamente nel rapporto 2:1 al raddoppiare del numero dei task paralleli, dovuta alla riduzione del numero di eventi che deve processare ciascun operatore.

V. VISUALIZZAZIONE DATI GRAFANA

La parte di data serving consente di visualizzare l'andamento dei voli statunitensi in tempo reale. La dashboard è stata divisa in due tab, uno per ciascuna query. Le query di estrazione e manipolazione dei dati sono state implementate tramite il linguaggio Flux. All'interno del database è stato creato un bucket per ciascuna query.

I dati della prima query sono stati strutturati all'interno del data store con:

- un *tag* relativo alla compagnia aerea, utilizzato per raggruppare i dati più facilmente;
- il *timestamp* corrispondente all'istante di inizio della finestra temporale all'interno della quale sono aggregati di dati;
- gli altri campi del risultato della query, escluso il timestamp di fine della finestra temporale, salvati come *field*.

Grafana mostra un grafico per ciascun *field*, con tracciati diversi per compagnia aerea. I dati sono estratti mediante semplici *filter*. I grafici scelti per la query sono:

- Time series per il ritardo medio in partenza, il numero dei voli totali, il numero dei voli completati, il numero dei voli devianti e il numero dei voli cancellati. Grafana richiede che i valori siano aggregati tramite *aggregateWindow* per gestire la quantità dei dati nella renderizzazione grafica. La finestra di aggregazione è stabilita dinamicamente da Grafana, utilizzando come strategia di aggregazione il massimo;
- State timeline per il tasso di voli con ritardo in partenza maggiore o uguale di quindici minuti. Lo stato associato a ciascuna compagnia aerea si colora di verde se il tasso è compreso tra 0 e 0.1, di giallo se il tasso è compreso tra 0.1 e 0.2 e di rosso se il tasso è compreso tra 0.2 e 1;
- Gauge per il tasso di voli cancellati.



Figura 3. Grafici relativi all'andamento del ritardo medio in partenza e del numero dei voli totali, dei voli completati e dei voli devianti, suddivisi per compagnia aerea. I valori delle immagini sono solo rappresentativi

Il modello dati per i risultati della seconda query è stato strutturato utilizzando due *measurement* distinte: la prima dedicata alla classifica generale degli aeroporti e la seconda al dettaglio dei singoli voli in ritardo per ciascun aeroporto. Questa scelta è dettata dai vincoli strutturali di InfluxDB, il quale supporta esclusivamente tipi di dato primitivi per i *field*. Pertanto, per storicizzare le due tipologie di classifiche dei

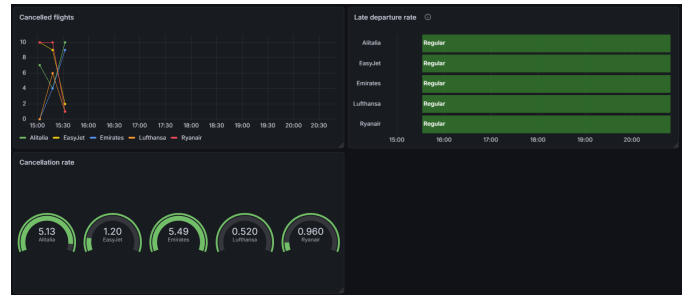


Figura 4. Grafici relativi all'andamento del numero di voli cancellati, del tasso di voli con ritardo in partenza maggiore o uguale di 15 minuti e del tasso di cancellazione, suddivisi per compagnia aerea. I valori delle immagini sono solo rappresentativi

voli (degli aeroporti e dei voli per ciascun aeroporto), si è resa necessaria una separazione delle due entità, salvando i voli come punti temporali distinti, ma logicamente collegati al rispettivo aeroporto di partenza.

I dati relativi agli aeroporti sono salvati utilizzando un *measurement* apposito strutturato nel modo seguente:

- due *tag*, corrispondenti alla dimensione della finestra temporale rispetto alla quale sono calcolati i risultati, e all'aeroporto di partenza;
- il *timestamp* corrispondente all'istante di inizio della finestra temporale all'interno della quale sono aggregati di dati;
- gli altri campi sono salvati come *field*.

I dati relativi ai voli delle classifiche sono salvati utilizzando un altro *measurement* specifico, strutturato nel modo seguente:

- tre *tag*, corrispondenti (i) alla dimensione della finestra temporale e (ii) all'aeroporto di origine, per assegnare il punto temporale all'aeroporto e alla finestra temporale corretti, (iii) la posizione in classifica, fondamentale per evitare che voli diversi appartenenti alla stessa finestra temporale e allo stesso aeroporto siano considerati come aggiornamenti nel tempo dello stesso punto temporale;
- il *timestamp* corrispondente all'istante di inizio della finestra temporale all'interno della quale sono aggregati di dati;
- gli altri campi sono salvati come *field*.

relativo ai voli delle classifiche è caratterizzato da tre *tag*:

In Grafana il tab della seconda query è suddiviso in base alla dimensione della finestra temporale attraverso l'utilizzo delle *Row*. All'interno di ciascuna *Row* sono presenti la classifica degli aeroporti, aggiornata al valore corrente, e la classifica dei voli partiti da uno specifico aeroporto della classifica, impostato come variabile della dashboard Grafana e selezionabile tramite apposito menu a tendina in alto a sinistra. Le classifiche sono state generate mediante l'utilizzo di *filter* e *pivot* per avere una struttura dei dati tabellare.

DICHIARAZIONE SULL'USO DI IA GENERATIVA

L'intelligenza artificiale generativa è stata utilizzata per le attività di debugging e brainstorming relativamente alla

progettazione dell’ambiente di lavoro. Il brainstorming si è rivelato particolarmente utile nell’implementazione della Query 2, portando alla scelta di utilizzare una coda di priorità per mantenere e aggiornare lo stato delle classifiche.

The screenshot shows a Tableau dashboard with two main tables. The first table, titled 'Top 10 origin airport per severe delays', lists the top 10 airports based on the number of flights with severe delays. The second table, titled 'Top 20 flights ordered by departure delay', lists the top 20 flights based on their departure delay.

Timestamp	rank	airport	Number of flights	Severe delays	Maximum departure delay	Mean departure delay
2028-07-05 16:58:20.291	1	MAD	229	40	82.4	37.9
2028-07-05 16:58:20.291	2	IST	472	16	82.2	85.5
2028-07-05 16:58:20.291	3	DXB	167	23	96.4	34.1
2028-07-05 16:58:20.291	3	CDG	368	20	70	28.6
2028-07-05 16:58:20.291	4	JFK	109	38	78.3	41.4
2028-07-05 16:58:20.291	4	LAX	121	36	80.5	32.9
2028-07-05 16:58:20.291	5	FRA	105	16	71.5	40.5
2028-07-05 16:58:20.291	6	AMS	120	18	85.9	30.3
2028-07-05 16:58:20.291	7	BCN	267	24	65.9	40.6
2028-07-05 16:58:20.291	8	MUC	351	29	58.8	25.3
2028-07-05 16:58:20.291	9	FCO	391	27	88.4	27.7
2028-07-05 16:58:20.291	10	LHR	188	30	83.4	43.2

origin_airport	rank	airline	Departure delay	Airport of destination
AMS	1	Easyjet	88.8	IST
AMS	2	Easyjet	78.6	MUC

Figura 5. Grafici relativi alla classifica dei dieci aeroporti con il maggior numero di voli con ritardo in partenza significativo e alla classifica dei voli con maggior ritardo in partenza significativo, calcolati su finestre temporali di un’ora. I valori sono solo rappresentativi

The screenshot shows a Tableau dashboard with two main tables. The first table, titled 'Top 10 origin airport per severe delays - 6H window', lists the top 10 airports based on the number of flights with severe delays over a 6-hour window. The second table, titled 'Top 20 flights ordered by departure delay', lists the top 20 flights based on their departure delay over a 6-hour window.

Timestamp	rank	airport	Number of flights	Severe delays	Maximum departure delay	Mean departure delay
2028-07-05 16:58:20.291	1	BCN	376	26	100	24.0
2028-07-05 16:58:20.291	2	DXB	429	9	45.6	40.2
2028-07-05 16:58:20.291	3	FRA	435	39	109	34.0
2028-07-05 16:58:20.291	4	JFK	110	25	63.0	29.9
2028-07-05 16:58:20.291	5	AMS	395	14	100	15.9
2028-07-05 16:58:20.291	6	LAX	158	40	75.9	32.7
2028-07-05 16:58:20.291	7	IST	123	5	104	37.1
2028-07-05 16:58:20.291	7	CDG	462	39	60.8	41.0
2028-07-05 16:58:20.291	8	LHR	150	34	54.6	22.7
2028-07-05 16:58:20.291	9	MUC	308	25	60.6	16.4
2028-07-05 16:58:20.291	10	MAD	278	33	103	39.4
2028-07-05 16:58:20.291	10	FCO	464	22	75.7	42.2

origin_airport	rank	airline	Departure delay	Airport of destination
AMS	1	Emirates	75.7	MAD
AMS	2	Lufthansa	101	JFK
AMS	3	Delta	69.1	AMS
AMS	4	Emirates	94.7	JFK

Figura 6. Grafici relativi alla classifica dei dieci aeroporti con il maggior numero di voli con ritardo in partenza significativo e alla classifica dei voli con maggior ritardo in partenza significativo, calcolati su finestre temporali di sei ore. I valori sono solo rappresentativi

The screenshot shows a Tableau dashboard with two main tables. The first table, titled 'Top 10 origin airport per severe delays - Global window', lists the top 10 airports based on the number of flights with severe delays globally. The second table, titled 'Top 20 flights ordered by departure delay', lists the top 20 flights based on their departure delay globally.

Timestamp	rank	airport	Number of flights	Severe delays	Maximum departure delay	Mean departure delay
2028-07-05 16:58:20.291	1	DXB	175	42	82.6	24.0
2028-07-05 16:58:20.291	2	MAD	257	26	48.1	22.5
2028-07-05 16:58:20.291	3	LAX	406	5	78.0	31.5
2028-07-05 16:58:20.291	4	BCN	308	16	48.4	23.8
2028-07-05 16:58:20.291	5	FRA	236	45	75.7	22.8
2028-07-05 16:58:20.291	6	MUC	444	19	66.8	31.9
2028-07-05 16:58:20.291	7	JFK	214	5	67.6	28.4
2028-07-05 16:58:20.291	7	CDG	19	41	96	28.9
2028-07-05 16:58:20.291	8	AMS	437	15	104	34.7
2028-07-05 16:58:20.291	9	FCO	197	5	67.9	24.2
2028-07-05 16:58:20.291	9	LHR	481	19	86.6	28.4
2028-07-05 16:58:20.291	10	IST	207	15	96.5	42.9

origin_airport	rank	airline	Departure delay	Airport of destination
AMS	1	Lufthansa	63.2	MUC
AMS	2	Delta	90.2	FRA
AMS	3	Ryanair	91.0	AMS
AMS	4	Easyjet	96.0	CDG
AMS	5	Ryanair	75.4	BCN

Figura 7. Grafici relativi alla classifica dei dieci aeroporti con il maggior numero di voli con ritardo in partenza significativo e alla classifica dei voli con maggior ritardo in partenza significativo, calcolati dall’inizio del dataset. I valori sono solo rappresentativi